

Programming in Python

Unit-I

Introduction to Python Programming

Additional Points:

Python Basics: Entering Expressions into the Interactive Shell, The Integer, Floating-Point, and String Data Types

Entering Expressions into the Interactive Shell:

The Integer Data Type (`int`):

The Floating-Point Data Type (`float`):

The String Data Type (`str`):

Additional Points:

String Concatenation

Concatenation:

String Replication

Replication:

Storing Values in Variables

Variables:

Dissecting Your Program

Understanding Your Program:

Flow Control

Boolean Values:

Comparison Operators:

Boolean Operators:

Mixing Boolean and Comparison Operators

Elements of Flow Control

Program Execution

Definition:

Steps in Program Execution:

Flow Control Statements

Types of Flow Control Statements:

Importing Modules

Ways to Import Modules:

Standard Library vs. Third-Party Modules:

Ending a Program Early with `sys.exit()`

Unit - II

Functions

Syntax of defining a function with parameters:

Example 1: Function without Parameters

Example 2: Function with Parameters

Example 3: Function with Multiple Parameters

Example 4: Function with Default Parameters

Example 5: Function with Keyword Arguments

Example 6: Function with Variable Number of Arguments

Example 7: Function with Keyword Arguments and Variable Number of Arguments

Return Values and Return Statements

The None Value

Keyword Arguments

print() Function

Local and Global Scope

Local Scope:

Global Scope:

The global Statement

Exception Handling

try-except Blocks

Handling Specific Exceptions

Raising Exceptions

Custom Exception Classes

Exception Propagation

Handling Exceptions in File Operations

Lists

The List Data Type

Working with Lists

Accessing Elements:

Slicing:

List Methods:

Adding Elements:

Removing Elements:

Searching and Counting:

Sorting and Reversing:

Augmented Assignment Operators:

Dictionaries and Structuring Data: The Dictionary Data Type

Pretty Printing

Using Data Structures to Model Real-World Things

Real-World Application:

Manipulating Strings

Working with Strings:

String Concatenation:

String Slicing:

Useful String Methods:

Unit-I

Introduction to Python Programming

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming. It has a large and comprehensive standard library, making it suitable for various applications, including web development, data analysis, artificial intelligence, and scientific computing.

1. Introduction to Python:

- History and features of Python.
- Python philosophy: "Readability counts."
- Python's role in modern programming paradigms.

2. Installing Python:

- Different distributions: CPython, Anaconda, etc.
- Installation steps on various operating systems: Windows, macOS, Linux.

3. Python Interpreter:

- Understanding the Python interpreter.
- Interactive mode vs. script mode.

4. Python Development Environment:

- IDEs (Integrated Development Environments): PyCharm, VS Code, Jupyter Notebook, etc.
- Text editors: Sublime Text, Atom, Vim, etc.

- Online platforms: Google Colab, Jupyter Online, etc.

5. First Python Program:

- Writing and executing a simple "Hello, World!" program.
- Understanding the structure of a Python program.

6. Python Syntax:

- Indentation: Python's use of whitespace for block delimiters.
- Comments: Single-line and multi-line comments.
- Naming conventions: Variables, functions, classes, and modules.

7. Data Types in Python:

- Numeric types: int, float, complex.
- Sequence types: list, tuple, range.
- Text type: str.
- Boolean type: bool.

8. Variables and Constants:

- Declaring and initializing variables.
- Naming rules and conventions.
- Constants and their significance.

9. Keywords and Identifiers:

- Reserved words in Python.
- Naming rules for identifiers.
- Best practices for choosing meaningful names.

10. Basic Input and Output:

- Using `input()` function for user input.
- Printing output using `print()` function.
- Formatting output with f-strings and `format()` method.

Additional Points:

- **Python Versions:** Python has two major versions in use today: Python 2.x and Python 3.x. Python 3.x is the current version and recommended for new development due to its many improvements over Python 2.x.
- **Community and Resources:** Python has a vast and active community with abundant resources, including documentation, tutorials, forums, and libraries.
- **Zen of Python:** A set of guiding principles for writing computer programs in Python. Accessible via `import this`.
- **Dynamic Typing:** Python is dynamically typed, meaning you don't need to declare variable types explicitly.
- **Interoperability:** Python can be easily integrated with other languages like C/C++, Java, and .NET.
- **Portability:** Python code is highly portable across different platforms and operating systems.

These topics provide a solid foundation for understanding Python programming, setting the stage for deeper exploration into its various features and capabilities.

Python Basics: Entering Expressions into the Interactive Shell, The Integer, Floating-Point, and String Data Types

Entering Expressions into the Interactive Shell:

- Python provides an interactive shell that allows users to enter expressions directly and see the results immediately.
- To launch the interactive shell, open the command prompt or terminal and type `python`.
- Once in the shell, you can enter expressions like mathematical operations (`+`, `-`, `*`, `/`), assignments (`=`), etc.
- The interactive shell is a great way to experiment with Python syntax and test small code snippets.

The Integer Data Type (`int`):

- Integers represent whole numbers without any fractional part.
- Examples of integers: `5`, `10`, `0`.
- Python supports operations such as addition, subtraction, multiplication, division, and modulus (`%`) on integers.
- Integers in Python can be of arbitrary size, allowing you to work with very large numbers without overflow errors.

The Floating-Point Data Type (`float`):

- Floating-point numbers represent real numbers with a fractional part.
- Examples of floating-point numbers: `3.14`, `0.001`, `2.0`.
- Python uses the `float` data type to represent floating-point numbers.
- Floating-point numbers can also be expressed in scientific notation, e.g., `6.022e23`.
- Arithmetic operations like addition, subtraction, multiplication, and division can be performed on floating-point numbers.

The String Data Type (`str`):

- Strings represent sequences of characters enclosed within single quotes (`'`) or double quotes (`"`).
- Examples of strings: `'hello'`, `"Python"`, `'123'`.
- Python provides many operations and methods for working with strings, including concatenation (`+`), slicing, indexing, and formatting.
- Strings can be manipulated in various ways, such as converting case (`upper()`, `lower()`), finding substrings (`find()`, `index()`), and replacing (`replace()`).
- Python also supports raw strings (`r'raw string'`) and multi-line strings (`'''multi-line string'''`).

Example:

```
# Integer
x = 5
print(x) # Output: 5
print(type(x)) # Output: <class 'int'>

# Floating-point
y = 3.14
print(y) # Output: 3.14
print(type(y)) # Output: <class 'float'>

# String
s = 'hello'
print(s) # Output: hello
print(type(s)) # Output: <class 'str'>
```

Additional Points:

- Python supports other numeric types such as complex numbers (`complex`) for working with complex arithmetic.
- String literals can also be enclosed within triple quotes (`'''` or `"""`) to create multi-line strings.
- In Python, strings are immutable, meaning once created, they cannot be modified. Any operation that appears to modify a string actually creates a new string object.
- Python provides many built-in functions and methods for performing operations on strings and converting between different data types.

Understanding these fundamental data types is crucial for building more complex Python programs and applications. They form the building blocks upon which more advanced concepts and techniques are built.

String Concatenation

Concatenation:

- **Definition:** Concatenation is the process of combining two or more strings into a single string.
- **Operator:** Use the `+` operator to concatenate strings.
- **Example:**

```
str1 = "Hello"
str2 = "World"
result_concatenation = str1 + str2
print("Concatenation:", result_concatenation) # Output: HelloWorld
```

Understanding string concatenation is essential for combining strings in Python to create meaningful outputs or manipulate text data effectively.

String Replication

Replication:

- **Definition:** Replication is the process of repeating a string multiple times.
- **Operator:** Use the `*` operator followed by an integer to replicate a string.
- **Example:**

```
str1 = "Python"
result_replication = str1 * 3
print("Replication:", result_replication) # Output: PythonPythonPython
```

Understanding string replication allows you to create repeated patterns or duplicate strings as needed in your Python programs.

Storing Values in Variables

Variables:

- **Definition:** Variables are named containers used to store data in memory.
- **Assignment:** Use the assignment operator (`=`) to assign a value to a variable.
- **Example:**

```
x = 10
name = "John"
print("Value of x:", x)          # Output: 10
print("Value of name:", name)    # Output: John
```

Understanding how to store values in variables allows you to manage data efficiently and perform operations on them throughout your Python program.

Dissecting Your Program

Understanding Your Program:

- **Comments:** Add comments to explain the purpose or functionality of different parts of your code.
- **Debugging:** Use `print()` statements to display intermediate results and debug your program.
- **Indentation:** Python uses indentation to define blocks of code, ensuring proper structure and readability.
- **Example:**

```
# This is a comment explaining the purpose of the code below
result = x * 2 # Multiply x by 2 and store the result in the variable 'result'
print("Result:", result) # Output: 20
```

Dissecting your program involves understanding its components, including comments, debugging techniques, indentation, and ensuring readability and correctness. This ensures that your code is clear, well-structured, and easy to maintain.

Flow Control

Boolean Values:

- **Definition:** Boolean values represent truth values - `True` or `False`.
- **Example:**

```
is_student = True
is_working = False
print("Is student?", is_student) # Output: True
print("Is working?", is_working) # Output: False
```

Boolean values are fundamental in Python for making decisions and controlling the flow of execution in programs.

Comparison Operators:

- **Definition:** Comparison operators are used to compare values and return a Boolean result.
- **Examples:**

```
x = 10
y = 20
print("x == y:", x == y) # Output: False (Equal)
print("x != y:", x != y) # Output: True (Not Equal)
print("x < y:", x < y)    # Output: True (Less Than)
print("x > y:", x > y)    # Output: False (Greater Than)
print("x <= y:", x <= y)  # Output: True (Less Than or Equal)
print("x >= y:", x >= y)  # Output: False (Greater Than or Equal)
```

Comparison operators allow you to compare values and make decisions based on the results in your Python programs.

Boolean Operators:

- **Definition:** Boolean operators (`and` , `or` , `not`) are used to combine or manipulate Boolean values.
- **Examples:**

```
a = True
b = False
print("a and b:", a and b)    # Output: False (Logical AND)
print("a or b:", a or b)      # Output: True (Logical OR)
print("not a:", not a)        # Output: False (Logical NOT)
```

Boolean operators are crucial for combining multiple conditions and controlling the flow of execution based on Boolean expressions in Python programs.

Understanding boolean values, comparison operators, and boolean operators is essential for building logical and efficient flow control mechanisms in Python programs.

Mixing Boolean and Comparison Operators

Example 1: Checking if a Number is Even and Greater than 10

```
num = 12
is_even_and_greater_than_10 = (num % 2 == 0) and (num > 10)
print("Is num even and greater than 10?", is_even_and_greater_than_10) # Output: True
```

In this example, we use the modulus operator `%` to check if the number is even (`num % 2 == 0`) and the greater than operator `>` to check if the number is greater than 10. We combine these conditions using the logical `and` operator to determine if the number satisfies both conditions simultaneously.

Example 2: Checking if a Number is Odd or Less than 5

```
num = 3
is_odd_or_less_than_5 = (num % 2 != 0) or (num < 5)
```

```
print("Is num odd or less than 5?", is_odd_or_less_than_5) #  
Output: True
```

Here, we use the inequality operator `!=` to check if the number is odd (`num % 2 != 0`) and the less than operator `<` to check if the number is less than 5. We combine these conditions using the logical `or` operator to determine if the number satisfies either of the conditions.

Example 3: Checking if a Number is not Equal to 0 and not Greater than 100

```
num = 75  
not_equal_to_0_and_not_greater_than_100 = (num != 0) and not  
(num > 100)  
print("Is num not equal to 0 and not greater than 100?", not_  
equal_to_0_and_not_greater_than_100) # Output: True
```

In this example, we first check if the number is not equal to 0 (`num != 0`) and then use the greater than operator `>` combined with the logical `not` operator to check if the number is not greater than 100. We use the logical `and` operator to combine these conditions to determine if the number satisfies both conditions simultaneously.

Mixing boolean and comparison operators allows for more complex condition checking in Python programs, enabling you to build robust and flexible logic for controlling the flow of execution based on various conditions.

Elements of Flow Control

Flow control refers to the order in which statements are executed in a program. It allows programs to make decisions, repeat code blocks, and execute different paths based on conditions.

Key Elements:

1. Sequence:

- **Description:** The default flow of control where statements are executed line by line in the order they appear in the program.
- **Example:**

```
print("Statement 1")
print("Statement 2")
print("Statement 3")
```

2. Selection:

- **Description:** Making decisions based on conditions using if statements.
- **Example:**

```
x = 10
if x > 5:
    print("x is greater than 5")
```

3. Iteration:

- **Description:** Repeating code blocks using loops like while and for loops.
- **Example (while loop):**

```
i = 0
while i < 5:
    print(i)
    i += 1
```

- **Example (for loop):**

```
for i in range(5):
    print(i)
```

4. Jump:

- **Description:** Altering the flow of control using break, continue, and return statements.
- **Example (break):**

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

- **Example (continue):**

```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```

- **Example (return):**

```
def add(x, y):  
    return x + y  
  
result = add(5, 3)  
print("Result:", result)
```

Understanding these elements is essential for controlling the flow of execution in Python programs and writing code that behaves as intended.

Program Execution

Definition:

Program execution refers to the process of running a program's code and producing the desired output.

Steps in Program Execution:

1. **Parsing:**

- **Description:** The interpreter reads the source code and checks for syntax errors.

2. Compilation:

- **Description:** The interpreter converts the source code into bytecode (for Python).

3. Execution:

- **Description:** The bytecode is executed by the Python Virtual Machine (PVM) line by line.

Example:

```
# Python program execution example
def greet(name):
    print("Hello, ", name)

greet("John")
```

Understanding program execution helps developers diagnose and troubleshoot issues in their code and optimize performance.

Flow Control Statements

Flow control statements are used to alter the flow of execution in a program based on certain conditions.

Types of Flow Control Statements:

1. if-elif-else:

- **Description:** Executes different blocks of code based on the evaluation of conditions.
- **Example:**

```
x = 10
if x > 5:
    print("x is greater than 5")
elif x == 5:
    print("x is equal to 5")
```

```
else:  
    print("x is less than 5")
```

2. while:

- **Description:** Repeats a block of code as long as the specified condition is true.
- **Example:**

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

3. for:

- **Description:** Iterates over a sequence (e.g., list, tuple, string) and executes a block of code for each item.
- **Example:**

```
for i in range(5):  
    print(i)
```

4. break:

- **Description:** Terminates the loop prematurely and exits the loop.
- **Example:**

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

5. continue:

- **Description:** Skips the current iteration of the loop and continues with the next iteration.
- **Example:**

```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```

6. **pass:**

- **Description:** Placeholder statement that does nothing when executed, used to avoid syntax errors when a statement is required syntactically but no action is needed.
- **Example:**

```
x = 10  
if x > 5:  
    pass # Placeholder for future code implementation
```

Understanding flow control statements enables developers to create more dynamic and flexible programs by controlling the flow of execution based on different conditions.

These elements are fundamental concepts in Python programming, allowing developers to create complex and sophisticated programs by controlling the flow of execution effectively.

Importing Modules

In Python, modules are files containing Python code that can be imported and used in other Python programs. Importing modules allows you to access functions, classes, and variables defined in those modules.

Ways to Import Modules:

1. Using import statement:

- Syntax: `import module_name`
- Example:

```
import math
print(math.sqrt(25)) # Output: 5.0
```

2. Using from...import statement:

- Syntax: `from module_name import function_name, class_name`
- Example:

```
from math import sqrt
print(sqrt(25)) # Output: 5.0
```

3. Using alias for modules:

- Syntax: `import module_name as alias`
- Example:

```
import math as m
print(m.sqrt(25)) # Output: 5.0
```

Standard Library vs. Third-Party Modules:

- **Standard Library:** Modules that come pre-installed with Python, providing a wide range of functionalities.
- **Third-Party Modules:** Modules developed by the Python community or third-party developers, which need to be installed separately using tools like pip.

Example (Using Standard Library Module - datetime):

```
import datetime
current_date = datetime.datetime.now()
print("Current date and time:", current_date)
```

Example (Using Third-Party Module - requests):

```
import requests
response = requests.get("<https://www.example.com>")
print("Response status code:", response.status_code)
```

Importing modules allows you to leverage existing code and extend the functionality of your Python programs by integrating additional features and capabilities.

Ending a Program Early with `sys.exit()`

The `sys.exit()` function is used to exit a Python program prematurely. It is commonly used to terminate the program when certain conditions are met or to handle exceptional cases.

Syntax:

```
import sys
sys.exit(exit_code)
```

- `exit_code` (Optional): An integer representing the exit status. Default is `0`, indicating successful termination.

Example:

```
import sys

def validate_input(input_value):
    if not input_value.isdigit():
        print("Invalid input. Please enter a valid integer.")
        sys.exit(1) # Exit with status code 1 (indicating error)

user_input = input("Enter an integer: ")
validate_input(user_input)
print("Valid input:", user_input)
```

In this example, if the user enters a non-integer value, the program exits early with a status code of `1`, indicating an error condition. This allows you to handle exceptional cases gracefully and provide appropriate feedback to the user.

Using `sys.exit()` enables you to control the termination of your Python programs effectively and handle various scenarios where premature termination is necessary.

Unit - II

Functions

Functions in Python are blocks of reusable code that perform a specific task. They allow you to break down your code into smaller, manageable parts and promote code reuse and modularity. Parameters are variables that are passed to a function to customize its behavior.

Syntax of defining a function with parameters:

```
def function_name(parameter1, parameter2, ...):  
    # Function body  
    # Code to perform a specific task using parameters  
    # Return statement (optional)
```

Example 1: Function without Parameters

```
def greet():  
    print("Hello, World!")  
  
# Calling the function without any parameters  
greet() # Output: Hello, World!
```

Example 2: Function with Parameters

```
def greet(name):  
    print("Hello,", name)  
  
# Calling the function with a parameter  
greet("John") # Output: Hello, John
```

Example 3: Function with Multiple Parameters

```
def add(x, y):  
    return x + y  
  
# Calling the function with two parameters  
result = add(5, 3)  
print("Result:", result) # Output: Result: 8
```

Example 4: Function with Default Parameters

```
def greet(name="Guest"):  
    print("Hello,", name)  
  
# Calling the function without passing a parameter  
greet() # Output: Hello, Guest  
  
# Calling the function with a parameter  
greet("Alice") # Output: Hello, Alice
```

Example 5: Function with Keyword Arguments

```
def greet(name, message):  
    print("Hello,", name + "!", message)  
  
# Calling the function with keyword arguments
```

```
greet(message="Have a nice day.", name="Alice") # Output: Hello, Alice! Have a nice day.
```

Example 6: Function with Variable Number of Arguments

```
def calculate_sum(*args):
    total = sum(args)
    return total

# Calling the function with variable number of arguments
result = calculate_sum(1, 2, 3, 4, 5)
print("Sum:", result) # Output: Sum: 15
```

Example 7: Function with Keyword Arguments and Variable Number of Arguments

```
def calculate_total(**kwargs):
    total = sum(kwargs.values())
    return total

# Calling the function with keyword arguments and variable number of arguments
result = calculate_total(apple=2, banana=3, orange=4)
print("Total fruits:", result) # Output: Total fruits: 9
```

Functions with parameters allow you to create flexible and reusable code that can be customized to suit different needs by accepting inputs and producing outputs based on those inputs.

Return Values and Return Statements

In Python, the `return` statement is used to exit a function and return a value to the caller. Functions can optionally return one or more values, which can be used for further computation or processing.

Syntax of the return statement:

```
def function_name(parameter1, parameter2, ...):  
    # Function body  
    # Code to perform a specific task  
    return value
```

Example 1: Function with a Return Statement

```
def add(x, y):  
    return x + y  
  
# Calling the function and storing the returned value  
result = add(5, 3)  
print("Result:", result) # Output: Result: 8
```

Example 2: Function with Multiple Return Statements

```
def absolute_value(x):  
    if x >= 0:  
        return x  
    else:  
        return -x  
  
# Calling the function and storing the returned value  
result1 = absolute_value(5)  
result2 = absolute_value(-3)  
print("Absolute value of 5:", result1) # Output: Absolute value of 5: 5  
print("Absolute value of -3:", result2) # Output: Absolute value of -3: 3
```

Example 3: Function with No Return Statement (Returns None)

```
def greet(name):  
    print("Hello, ", name)
```

```
# Calling the function and storing the returned value
result = greet("John")
print("Result:", result) # Output: Result: None
```

The None Value

In Python, `None` is a special constant that represents the absence of a value or a null value. It is often used to indicate that a variable or function does not have a meaningful value or has not been initialized.

Example 1: Assigning None to a Variable

```
x = None
print("Value of x:", x) # Output: Value of x: None
```

Example 2: Returning None from a Function

```
def greet(name):
    print("Hello,", name)

# Calling the function without storing the return value
result = greet("John")
print("Result:", result) # Output: Result: None
```

Example 3: Checking for None

```
def find_element(lst, target):
    for element in lst:
        if element == target:
            return element
    return None

my_list = [1, 2, 3, 4, 5]
result = find_element(my_list, 6)
if result is None:
    print("Element not found.")
```

```
else:
    print("Element found:", result)
```

Understanding return values and the `None` value allows you to write functions that can return meaningful results and handle cases where no value is returned or when a value is explicitly set to `None`.

Keyword Arguments

Keyword arguments are arguments passed to a function using the parameter names along with their corresponding values. This allows for more explicit and readable function calls, especially when dealing with functions that have multiple parameters.

Syntax of using keyword arguments:

```
def function_name(param1=value1, param2=value2, ...):
    # Function body
    # Code to perform a specific task
```

Example 1: Function Call with Keyword Arguments

```
def greet(name, message):
    print("Hello,", name + "!", message)

# Calling the function with keyword arguments
greet(message="Have a nice day.", name="Alice") # Output: Hello, Alice! Have a nice day.
```

Example 2: Function with Default Parameters and Keyword Arguments

```
def greet(name="Guest", message="Welcome!"):
    print("Hello,", name + "!", message)

# Calling the function without passing any arguments
greet() # Output: Hello, Guest! Welcome!
```

```
# Calling the function with keyword arguments
greet(message="Goodbye!", name="Alice") # Output: Hello, Alice! Goodbye!
```

Example 3: Mixing Positional and Keyword Arguments

```
def greet(name, message):
    print("Hello,", name + "!", message)

# Calling the function with a mix of positional and keyword arguments
greet("John", message="How are you?") # Output: Hello, John!
How are you?
```

Using keyword arguments allows for clearer function calls and improves the readability of your code, especially when dealing with functions that have many parameters.

print() Function

The `print()` function in Python is used to display text or variables to the standard output device, typically the console. It allows you to output strings, numbers, variables, and expressions.

Syntax of the `print()` function:

```
print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

- `value1`, `value2`, ...: Values or variables to be printed.
- `sep`: Separator between the values. Default is a space.
- `end`: Ending character(s) to be printed. Default is a newline character (`\n`).
- `file`: File object where the output will be printed. Default is `sys.stdout` (standard output).
- `flush`: Whether to forcibly flush the stream. Default is `False`.

Example 1: Printing a String

```
print("Hello, World!") # Output: Hello, World!
```

Example 2: Printing Variables

```
x = 10
y = 20
print("x =", x, "and y =", y) # Output: x = 10 and y = 20
```

Example 3: Changing the Separator

```
print("Python", "Programming", "Language", sep=' - ') # Output: Python-Programming-Language
```

Example 4: Changing the Ending Character

```
print("Hello,", end=' ')
print("World!") # Output: Hello, World!
```

Example 5: Redirecting Output to a File

```
with open('output.txt', 'w') as f:
    print("Hello, World!", file=f)
```

The `print()` function is a versatile tool for outputting information in Python, and understanding its various parameters allows for more flexible and customized output formatting.

Local and Global Scope

In Python, variables can have different scopes, determining where they can be accessed or modified within a program. The two primary scopes are local and global.

Local Scope:

- Variables defined inside a function have local scope.
- They can only be accessed within the function where they are defined.
- Attempting to access a local variable outside its function will result in a `NameError`.

Example:

```
def my_function():  
    local_var = 10  
    print(local_var)  # Accessing local_var within the function  
  
my_function()  # Output: 10  
# print(local_var)  # This will raise a NameError because local_var is not defined outside the function
```

Global Scope:

- Variables defined outside any function or in the global scope have global scope.
- They can be accessed and modified from anywhere in the program, including inside functions.
- Accessing a global variable inside a function does not require any special declaration.

Example:

```
global_var = 20  # Global variable  
  
def my_function():  
    print(global_var)  # Accessing global_var within the function  
  
my_function()  # Output: 20  
print(global_var)  # Output: 20
```

The global Statement

The `global` statement in Python is used to declare that a variable inside a function is referring to a global variable defined outside the function. It allows you to modify global variables from within functions.

Syntax:

```
global variable_name
```

Example:

```
x = 10 # Global variable

def my_function():
    global x # Declare x as global within the function
    x = 20 # Modify the global variable x
    print("Inside the function:", x)

my_function() # Output: Inside the function: 20
print("Outside the function:", x) # Output: Outside the function: 20
```

Without the `global` statement, modifying a variable inside a function creates a new local variable with the same name, leaving the global variable unchanged.

Example without global statement:

```
x = 10 # Global variable

def my_function():
    x = 20 # This creates a new local variable x
    print("Inside the function:", x)

my_function() # Output: Inside the function: 20
print("Outside the function:", x) # Output: Outside the function: 10 (global variable remains unchanged)
```

Understanding local and global scope, as well as using the `global` statement when necessary, is essential for writing modular and maintainable code in Python.

Exception Handling

Exception handling in Python allows you to gracefully manage and respond to errors and exceptional situations that may occur during the execution of your program. By handling exceptions, you can prevent your program from crashing and provide meaningful error messages or alternative behaviors.

try-except Blocks

The primary mechanism for handling exceptions in Python is the `try-except` block.

Syntax:

```
try:
    # Code that may raise an exception
    # ...
except ExceptionType1:
    # Code to handle ExceptionType1
    # ...
except ExceptionType2 as variable:
    # Code to handle ExceptionType2
    # Use variable to access information about the exception
    # ...
except (ExceptionType3, ExceptionType4):
    # Code to handle multiple exceptions
    # ...
except:
    # Code to handle any other exceptions
    # ...
else:
    # Code to execute if no exceptions occur in the try block
    # ...
finally:
    # Optional block of code that always executes, regardless
```

```
of whether an exception occurred or not
    # Useful for cleanup or resource releasing tasks
    # ...
```

Example:

```
try:
    x = 10 / 0 # This will raise a ZeroDivisionError
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("Division successful.")
finally:
    print("Cleanup code.")
```

Handling Specific Exceptions

You can use specific `except` blocks to handle different types of exceptions separately.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ValueError:
    print("Invalid input. Please enter a valid integer.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Raising Exceptions

You can use the `raise` statement to explicitly raise exceptions in your code.

```
x = -1
if x < 0:
```

```
raise ValueError("x should be a non-negative number.")
```

Custom Exception Classes

You can define custom exception classes by subclassing from the built-in `Exception` class.

```
class MyCustomError(Exception):
    pass

try:
    raise MyCustomError("This is a custom error message.")
except MyCustomError as e:
    print("Custom error caught:", e)
```

Exception Propagation

If an exception is not caught within a function, it propagates up the call stack until it is caught or the program terminates.

Handling Exceptions in File Operations

When working with files, it's important to handle exceptions that may occur during file operations, such as opening, reading, writing, or closing files.

```
try:
    with open("example.txt", "r") as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("File not found.")
except IOError:
    print("Error reading the file.")
```

Exception handling in Python is a powerful mechanism for writing robust and reliable code. By using `try-except` blocks and other exception-related constructs,

you can gracefully handle errors and ensure that your programs behave predictably even in the face of unexpected situations.

Lists

In Python, a list is a versatile data structure that can store a collection of items. Lists are mutable, meaning their elements can be changed after the list is created. Lists are ordered, meaning the elements have a specific order, and they can contain duplicate elements.

Lists in Python are defined using square brackets `[]` and can contain elements separated by commas.

```
my_list = [1, 2, 3, 4, 5]
```

Elements in a list can be accessed using indexing. Python uses zero-based indexing, meaning the first element has an index of 0, the second element has an index of 1, and so on.

```
my_list = [10, 20, 30, 40, 50]
print(my_list[0]) # Output: 10
print(my_list[2]) # Output: 30
```

You can also access a subset of elements from a list using slicing.

```
my_list = [1, 2, 3, 4, 5]
print(my_list[1:4]) # Output: [2, 3, 4]
```

Python provides various methods to manipulate lists, such as `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `index()`, `count()`, `sort()`, and `reverse()`.

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list) # Output: [1, 2, 3, 4]

my_list.extend([5, 6, 7])
print(my_list) # Output: [1, 2, 3, 4, 5, 6, 7]
```

```
my_list.insert(2, 8)
print(my_list) # Output: [1, 2, 8, 3, 4, 5, 6, 7]

my_list.remove(3)
print(my_list) # Output: [1, 2, 8, 4, 5, 6, 7]

popped_item = my_list.pop()
print(popped_item) # Output: 7
print(my_list) # Output: [1, 2, 8, 4, 5, 6]

print(my_list.index(4)) # Output: 3

print(my_list.count(5)) # Output: 1

my_list.sort()
print(my_list) # Output: [1, 2, 4, 5, 6, 8]

my_list.reverse()
print(my_list) # Output: [8, 6, 5, 4, 2, 1]
```

List comprehensions provide a concise way to create lists in Python.

```
# Create a list of squares of numbers from 1 to 5
squares = [x**2 for x in range(1, 6)]
print(squares) # Output: [1, 4, 9, 16, 25]
```

Lists are fundamental data structures in Python, widely used for storing and manipulating collections of data. Understanding how to create, access, and manipulate lists is essential for effective Python programming.

The List Data Type

Lists in Python are a versatile and fundamental data structure used to store collections of items. Lists are mutable, meaning their elements can be changed after the list is created. They are ordered and allow duplicate elements.

Lists are defined using square brackets `[]` and can contain elements separated by commas.

```
my_list = [1, 2, 3, 4, 5]
```

Working with Lists

Once a list is created, you can perform various operations on it, such as accessing elements, modifying elements, adding or removing elements, and more.

Accessing Elements:

You can access individual elements of a list using indexing. Python uses zero-based indexing, where the first element has an index of 0, the second element has an index of 1, and so on.

```
my_list = [10, 20, 30, 40, 50]
print(my_list[0]) # Output: 10
print(my_list[2]) # Output: 30
```

Slicing:

You can also access a subset of elements from a list using slicing.

```
my_list = [1, 2, 3, 4, 5]
print(my_list[1:4]) # Output: [2, 3, 4]
```

List Methods:

Python provides a variety of built-in methods for working with lists. These methods allow you to manipulate the contents of a list in various ways.

Adding Elements:

- `append()` : Adds an element to the end of the list.
- `extend()` : Extends the list by appending elements from another list.

- `insert()` : Inserts an element at a specified position.

Removing Elements:

- `remove()` : Removes the first occurrence of a specified value.
- `pop()` : Removes the element at the specified position and returns it.

Searching and Counting:

- `index()` : Returns the index of the first occurrence of a specified value.
- `count()` : Returns the number of occurrences of a specified value.

Sorting and Reversing:

- `sort()` : Sorts the elements of the list in ascending order.
- `reverse()` : Reverses the order of the elements in the list.

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list) # Output: [1, 2, 3, 4]

my_list.extend([5, 6, 7])
print(my_list) # Output: [1, 2, 3, 4, 5, 6, 7]

my_list.insert(2, 8)
print(my_list) # Output: [1, 2, 8, 3, 4, 5, 6, 7]

my_list.remove(3)
print(my_list) # Output: [1, 2, 8, 4, 5, 6, 7]

popped_item = my_list.pop()
print(popped_item) # Output: 7
print(my_list) # Output: [1, 2, 8, 4, 5, 6]

print(my_list.index(4)) # Output: 3

print(my_list.count(5)) # Output: 1
```

```
my_list.sort()
print(my_list)  # Output: [1, 2, 4, 5, 6, 8]

my_list.reverse()
print(my_list)  # Output: [8, 6, 5, 4, 2, 1]
```

Lists are powerful and commonly used data structures in Python. By understanding how to create and manipulate lists, you gain a foundational skill that is essential for many programming tasks.

Augmented Assignment Operators:

Augmented assignment operators are shorthand operators in Python that combine an arithmetic or bitwise operation with an assignment statement. They provide a more concise way to modify the value of a variable based on its current value.

Examples of Augmented Assignment Operators:

Addition:

```
x = 5
x += 3  # Equivalent to x = x + 3
print(x)  # Output: 8
```

Subtraction:

```
x = 10
x -= 4  # Equivalent to x = x - 4
print(x)  # Output: 6
```

Multiplication:

```
x = 3
x *= 2  # Equivalent to x = x * 2
print(x)  # Output: 6
```

Division:

```
x = 12
x /= 3 # Equivalent to x = x / 3
print(x) # Output: 4.0
```

Modulus:

```
x = 15
x %= 4 # Equivalent to x = x % 4
print(x) # Output: 3
```

Exponentiation:

```
x = 2
x **= 3 # Equivalent to x = x ** 3
print(x) # Output: 8
```

Floor Division:

```
x = 17
x //= 5 # Equivalent to x = x // 5
print(x) # Output: 3
```

Augmented assignment operators provide a concise and efficient way to update the value of a variable based on its current value.

Methods:

In Python, methods are functions associated with objects. They are called using the dot notation on an object. Methods are used to perform operations on the object and can modify its state or return a value.

Example of a Method:

`append()` Method:

The

`append()` method is used to add an element to the end of a list.

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list) # Output: [1, 2, 3, 4]
```

Common List Methods:

Python provides several built-in methods for working with lists. Here are some commonly used list methods:

- `append()` : Adds an element to the end of the list.
- `extend()` : Extends the list by appending elements from another list.
- `insert()` : Inserts an element at a specified position.
- `remove()` : Removes the first occurrence of a specified value.
- `pop()` : Removes the element at the specified position and returns it.
- `index()` : Returns the index of the first occurrence of a specified value.
- `count()` : Returns the number of occurrences of a specified value.
- `sort()` : Sorts the elements of the list in ascending order.
- `reverse()` : Reverses the order of the elements in the list.

Example:

```
my_list = [3, 1, 4, 1, 5, 9, 2, 6, 5]
my_list.sort() # Sort the list in ascending order
print(my_list) # Output: [1, 1, 2, 3, 4, 5, 5, 6, 9]
```

Methods provide a convenient way to perform operations on objects in Python, enhancing the functionality and versatility of the language.

Dictionaries and Structuring Data: The Dictionary Data Type

Dictionaries in Python are a versatile data type used to store key-value pairs. They are mutable, unordered, and can contain duplicate keys (although keys must be

unique within one dictionary). Dictionaries are commonly used for representing structured data, such as records, mappings, or configurations.

Definition:

A dictionary in Python is defined using curly braces `{ }` and consists of key-value pairs separated by commas. Each key is separated from its corresponding value by a colon `:`.

Example:

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
```

Accessing Elements:

You can access the value associated with a specific key in a dictionary using square bracket notation or the `get()` method.

Example:

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
print(my_dict["name"]) # Output: John

print(my_dict.get("age")) # Output: 30
```

Adding and Modifying Elements:

You can add new key-value pairs to a dictionary or modify the values associated with existing keys.

Example:

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
my_dict["gender"] = "Male" # Adding a new key-value pair
print(my_dict) # Output: {'name': 'John', 'age': 30, 'city': 'New York', 'gender': 'Male'}

my_dict["age"] = 35 # Modifying the value associated with the 'age' key
```

```
print(my_dict) # Output: {'name': 'John', 'age': 35, 'city': 'New York', 'gender': 'Male'}
```

Removing Elements:

You can remove key-value pairs from a dictionary using the `del` keyword or the `pop()` method.

Example:

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
del my_dict["age"] # Removing the 'age' key-value pair
print(my_dict) # Output: {'name': 'John', 'city': 'New York'}

removed_value = my_dict.pop("city") # Removing the 'city' key-value pair using pop()
print(my_dict) # Output: {'name': 'John'}
print(removed_value) # Output: New York
```

Dictionaries are powerful data structures in Python, offering efficient ways to organize and manipulate data in key-value pairs. They are widely used in various applications due to their flexibility and ease of use.

Pretty Printing

Pretty printing in Python refers to formatting complex data structures, such as dictionaries or lists, in a visually appealing and readable way. It helps improve the readability of output for humans, especially when dealing with nested or large data structures.

Python provides the `pprint` module (pretty print) for this purpose. The `pprint` module offers a `pprint()` function that formats the output of data structures in a more readable and structured manner compared to the standard `print()` function.

Consider a nested dictionary that represents a configuration:

```
config = {
    'server': {
```

```

        'host': 'localhost',
        'port': 8080
    },
    'database': {
        'name': 'mydb',
        'user': 'admin',
        'password': 'password123'
    },
    'debug': True
}

```

Printing this dictionary using the `print()` function may result in a less readable output due to its nested structure:

```
print(config)
```

Output:

```

{'server': {'host': 'localhost', 'port': 8080}, 'database':
{'name': 'mydb', 'user': 'admin', 'password': 'password123'},
'debug': True}

```

Using the `pprint()` function from the `pprint` module provides a more organized and visually appealing output:

```

import pprint

pprint.pprint(config)

```

Output:

```

{'database': {'name': 'mydb', 'password': 'password123', 'use
r': 'admin'},
'debug': True,
'server': {'host': 'localhost', 'port': 8080}}

```

As you can see, `pprint()` formats the dictionary with indentation and line breaks, making it easier to read, especially for nested data structures.

Pretty printing is particularly useful when dealing with large or complex data structures, such as configuration files, JSON data, or deeply nested dictionaries and lists. It helps developers and users understand the structure of the data more easily, leading to improved code readability and maintainability.

Using Data Structures to Model Real-World Things

In programming, data structures play a crucial role in representing and modeling real-world entities or concepts. By choosing appropriate data structures, developers can efficiently store, organize, and manipulate data to reflect the characteristics and relationships of real-world objects.

Example:

Consider a simple example of modeling a student in a school using Python data structures.

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

# Creating instances of the Student class to represent individual students
student1 = Student("Alice", 15, "10th")
student2 = Student("Bob", 16, "11th")
student3 = Student("Charlie", 15, "10th")
```

In this example:

- We define a `Student` class with attributes `name`, `age`, and `grade`.
- We create instances of the `Student` class (`student1`, `student2`, `student3`) to represent individual students.

- Each student object encapsulates data about a specific student, such as their name, age, and grade.

Real-World Application:

In real-world applications, data structures are used to model various entities and scenarios:

1. **Customer Data in an E-commerce Platform:** Using dictionaries or objects to represent customers with attributes like name, address, email, and purchase history.
2. **Inventory Management System:** Using lists or dictionaries to track product inventory, with each item storing details like SKU, quantity, price, etc.
3. **Social Media Networks:** Modeling users and their relationships (e.g., friends, followers) using graphs or dictionaries, with each user storing profile information and connections to other users.
4. **Transportation System:** Representing vehicles, routes, and schedules using graphs or matrices to optimize transportation logistics.
5. **Healthcare Records:** Storing patient information, medical history, and treatment plans using dictionaries or database tables with well-defined schemas.

By leveraging appropriate data structures, developers can create efficient and scalable solutions that accurately model real-world scenarios. Additionally, understanding how to choose and manipulate data structures is essential for designing robust and maintainable software systems.

Manipulating Strings

Strings are fundamental data types in Python used to represent sequences of characters. Python provides a rich set of built-in functions and methods for manipulating strings, allowing developers to perform various operations such as concatenation, slicing, searching, and formatting.

Working with Strings:

String Concatenation:

Concatenation is the process of combining two or more strings into a single string.

Example:

```
str1 = "Hello"
str2 = "World"
result = str1 + ", " + str2
print(result) # Output: Hello, World
```

String Slicing:

Slicing allows you to extract a substring from a string by specifying a range of indices.

Example:

```
text = "Python Programming"
substring = text[0:6] # Extracts characters from index 0 to 5 (exclusive)
print(substring) # Output: Python
```

Useful String Methods:

`len()`:

Returns the length of the string.

Example:

```
text = "Hello, World!"
length = len(text)
print(length) # Output: 13
```

`lower()` and `upper()`:

Converts the string to lowercase or uppercase, respectively.

Example:

```
text = "Hello, World!"
print(text.lower()) # Output: hello, world!
print(text.upper()) # Output: HELLO, WORLD!
```

`strip()`:

Removes leading and trailing whitespace characters from the string.

Example:

```
text = "  Hello, World!  "
stripped_text = text.strip()
print(stripped_text) # Output: Hello, World!
```

`split()`:

Splits the string into a list of substrings based on a delimiter.

Example:

```
text = "apple,banana,orange"
fruits = text.split(",")
print(fruits) # Output: ['apple', 'banana', 'orange']
```

`join()`:

Concatenates elements of an iterable (e.g., list) into a single string, using the specified separator.

Example:

```
fruits = ['apple', 'banana', 'orange']
text = ", ".join(fruits)
print(text) # Output: apple, banana, orange
```

`find()` and `replace()`:

- `find()`: Searches for a substring within the string and returns its index. Returns -1 if the substring is not found.

- `replace()`: Replaces occurrences of a substring with another substring.

Example:

```
text = "Hello, World!"  
index = text.find("World")  
print(index) # Output: 7  
  
new_text = text.replace("World", "Python")  
print(new_text) # Output: Hello, Python!
```

Python's string manipulation capabilities are powerful and versatile, enabling developers to perform a wide range of tasks efficiently. Understanding these string methods is essential for working with textual data effectively in Python programs.

updated: <https://yashnote.notion.site/Programming-in-Python-e53a7b3ac75e4f73b6d982008e39f348?pvs=4>